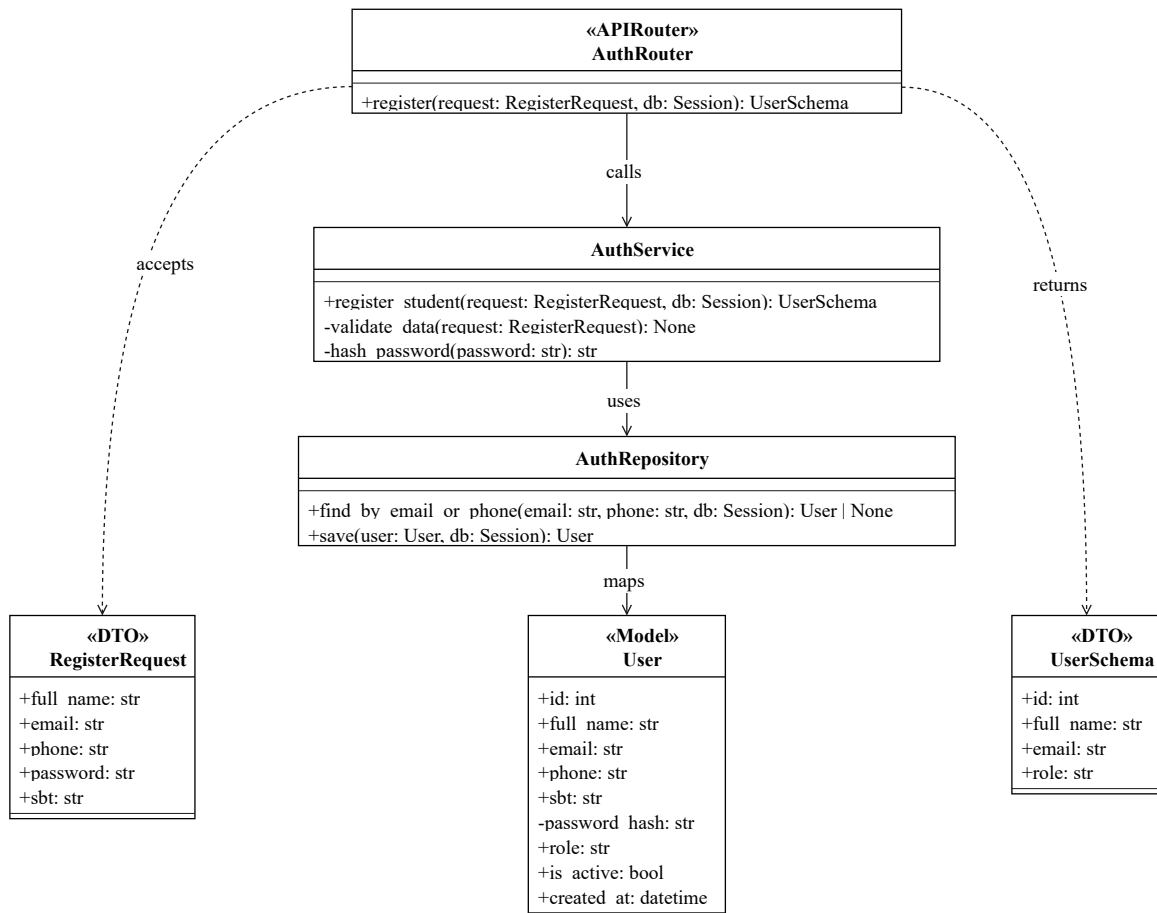


DRAFT REVIEW: DETAILED DESIGN

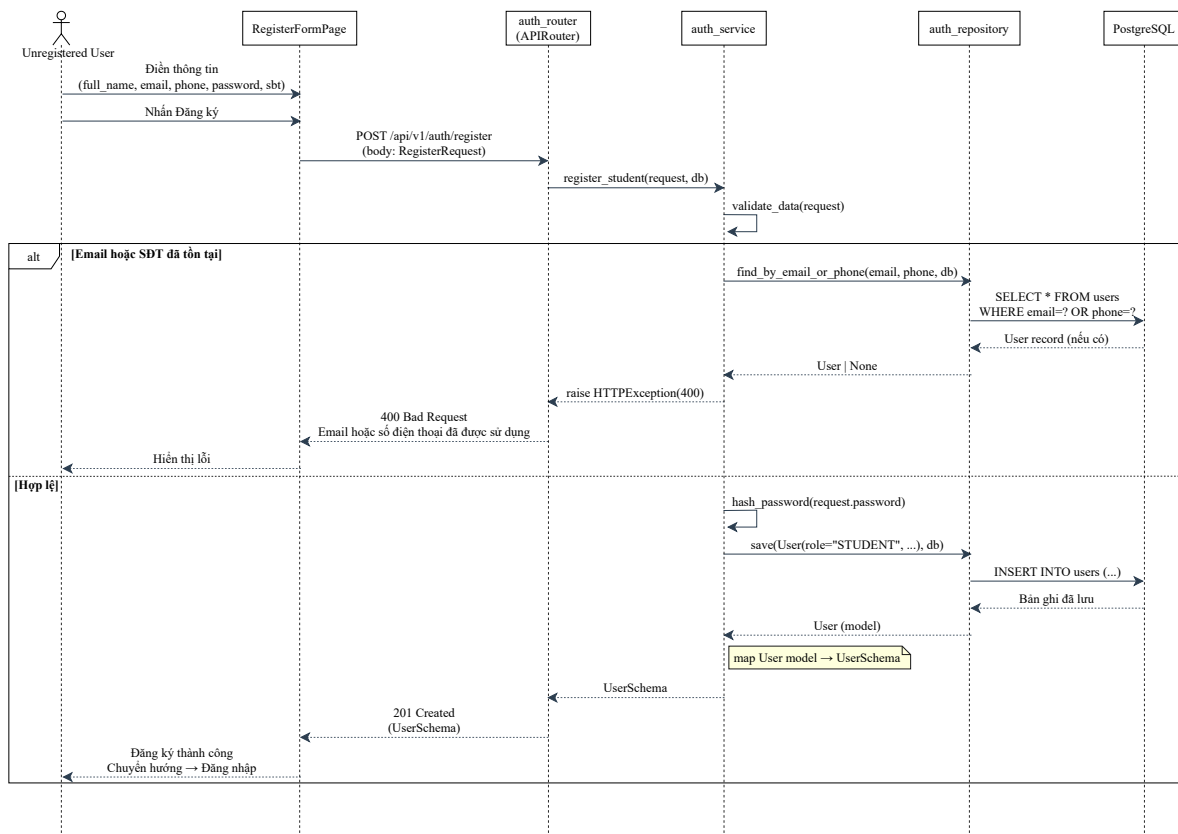
3.1 Log in (Đăng ký/Đăng nhập)

1. Class Diagram



Hình 1: Class_Diagram - Đăng Ký Tài Khoản

2. Sequence Diagram



Hình 2: Sequence_Diagram - Đăng ký tài khoản

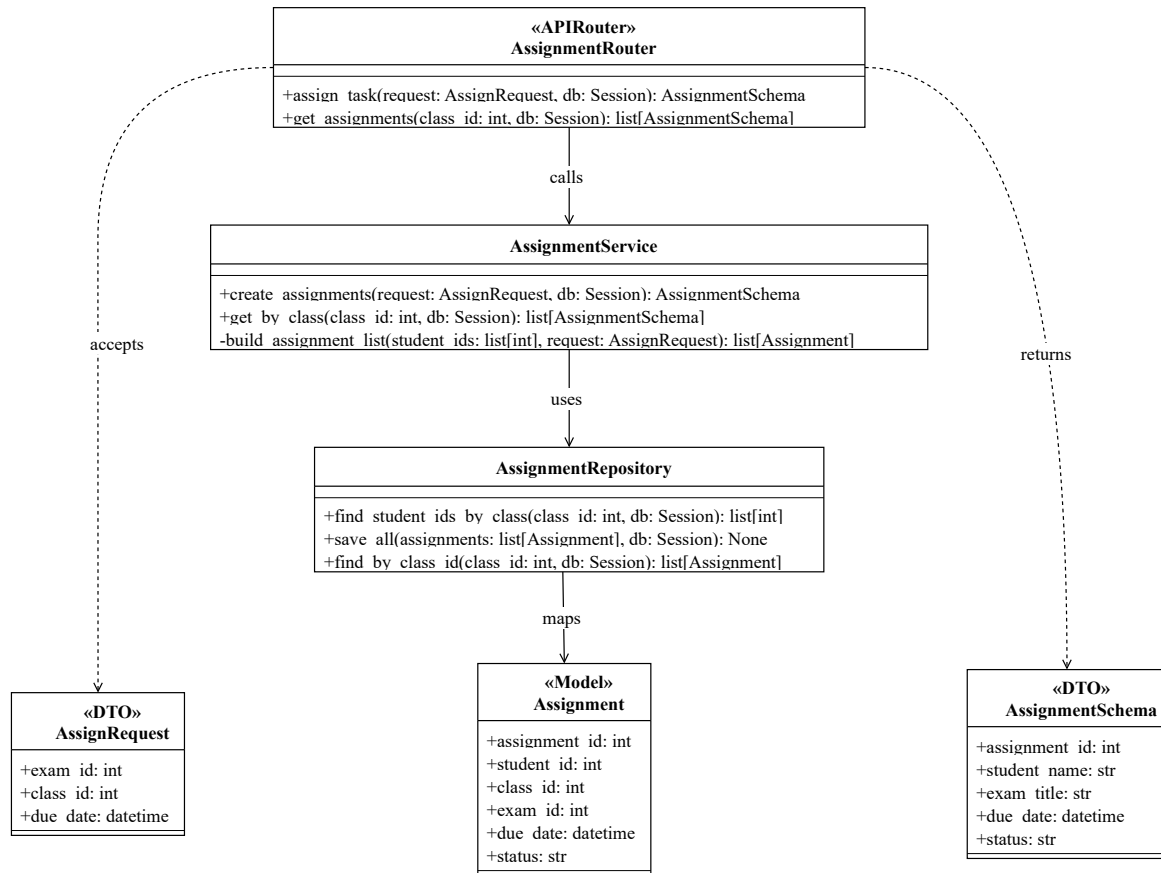
3. Class Specification

No	Class	Attributes / Methods	Description
1	User	id: int, full_name: str, email: str, phone: str, sbt: str, password_hash: str, role: str, is_active: bool, created_at: datetime	SQLAlchemy Model đại diện cho tài khoản người dùng trong bảng users. Lưu thông tin cá nhân, vai trò (STUDENT/TEACHER/PARENT/ADMIN) và trạng thái hoạt động của tài khoản.
2	RegisterRequest	full_name: str, email: str, phone: str, password: str, sbt: str	Pydantic Schema đầu vào chứa thông tin cần thiết để đăng ký tài khoản học sinh mới.

No	Class	Attributes / Methods	Description
3	UserSchema	id: int, full_name: str, email: str, role: str	Pydantic Schema đầu ra trả về sau khi đăng ký thành công, không bao gồm thông tin nhạy cảm (mật khẩu).
4	AuthRouter	register(request: RegisterRequest, db: Session): UserSchema	FastAPI APIRouter tiếp nhận HTTP POST request đăng ký và chuyển sang AuthService xử lý. Session được inject qua deps.get_db.
5	AuthService	register_student(request: RegisterRequest, db: Session): UserSchema, validate_data(request: RegisterRequest): None, hash_password(password: str): str	Service chứa toàn bộ business logic: kiểm tra dữ liệu đầu vào, băm mật khẩu và lưu tài khoản mới. Áp dụng BR-03 (mật khẩu được băm), BR-04 (gán vai trò STUDENT mặc định), BR-08 (email/SĐT không trùng).
6	AuthRepository	find_by_email_or_phone(email: str, phone: str, db: Session): User None, save(user: User, db: Session): User	Data access layer sử dụng SQLAlchemy ORM để kiểm tra trùng lặp và lưu bản ghi User vào PostgreSQL.

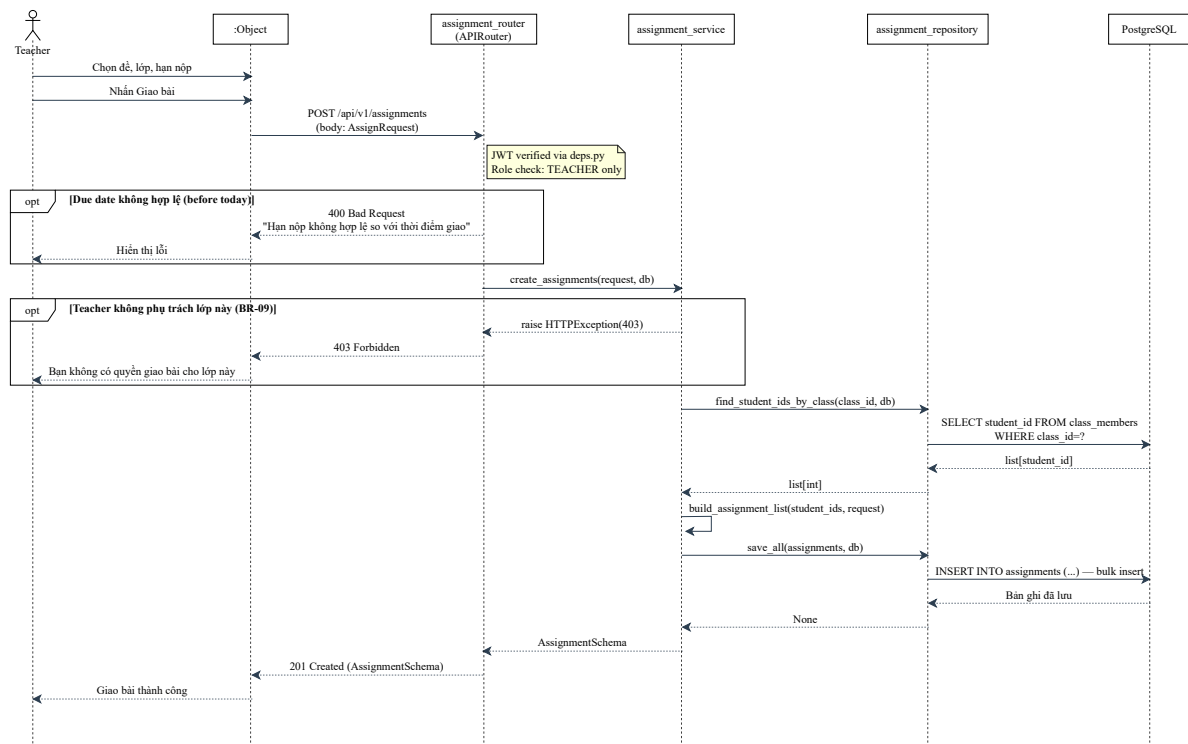
3.2 Assign an Assessment (Giao bài tập)

1. Class Diagram



Hình 3: Class_Diagram - Giáo viên giao bài tập

2. Sequence Diagram



Hình 4: Sequence_Diagram - Giáo viên giao bài tập

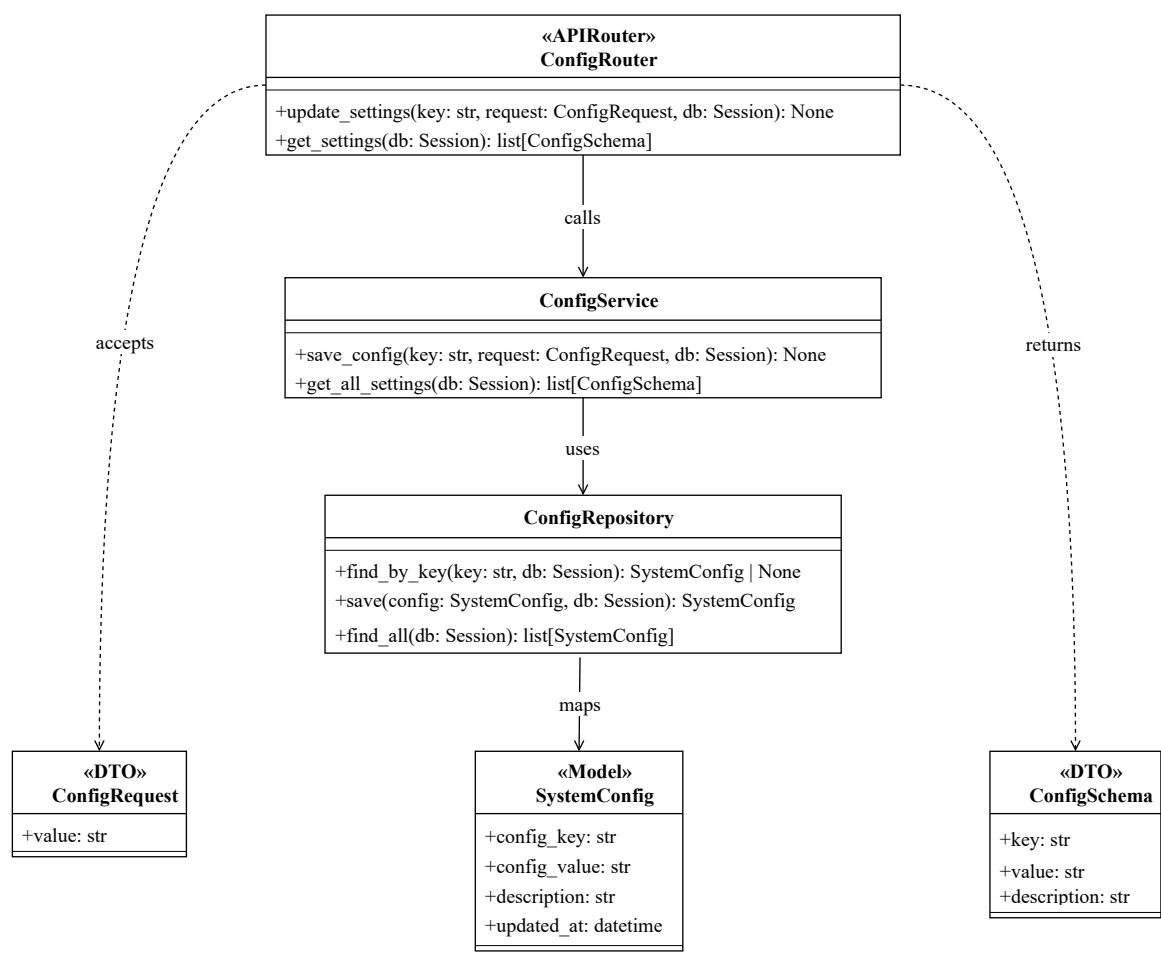
3. Class Specification

No	Class	Attributes / Methods	Description
1	Assignment	assignment_id: int, student_id: int, class_id: int, exam_id: int, due_date: datetime, status: str	SQLAlchemy Model lưu thông tin một bài tập được giao. Field class_id liên kết với lớp học, đảm bảo đúng BR-61 (bài giao phải gắn với lớp và học sinh hợp lệ).
2	AssignRequest	exam_id: int, class_id: int, due_date: datetime	Pydantic Schema đầu vào chứa thông tin giáo viên cung cấp khi giao bài tập cho cả lớp.
3	AssignmentSchema	assignment_id: int, student_name: str, exam_title: str, due_date: datetime, status: str	Pydantic Schema đầu ra trả về thông tin bài tập sau khi giao thành công.

No	Class	Attributes / Methods	Description
4	AssignmentRouter	assign_task(request: AssignRequest, db: Session): AssignmentSchema, get_assignments(class_id: int, db: Session): list[AssignmentSchema]	FastAPI APIRouter tiếp nhận POST /assignments để giao bài và GET /assignments để lấy danh sách bài đã giao theo lớp.
5	AssignmentService	create_assignments(request: AssignRequest, db: Session): AssignmentSchema, get_by_class(class_id: int, db: Session): list[AssignmentSchema], build_assignment_list(student_ids: list[int], request: AssignRequest): list[Assignment]	Service điều phối: lấy danh sách học sinh của lớp, tạo bản ghi Assignment cho từng học sinh, và lưu vào database. Áp dụng BR-62 (hạn nộp phải hợp lệ).
6	AssignmentRepository	find_student_ids_by_class(class_id: int, db: Session): list[int], save_all(assignments: list[Assignment], db: Session): None, find_by_class_id(class_id: int, db: Session): list[Assignment]	Data access layer lấy danh sách student theo lớp, lưu nhiều bản ghi Assignment cùng lúc và truy vấn bài đã giao.

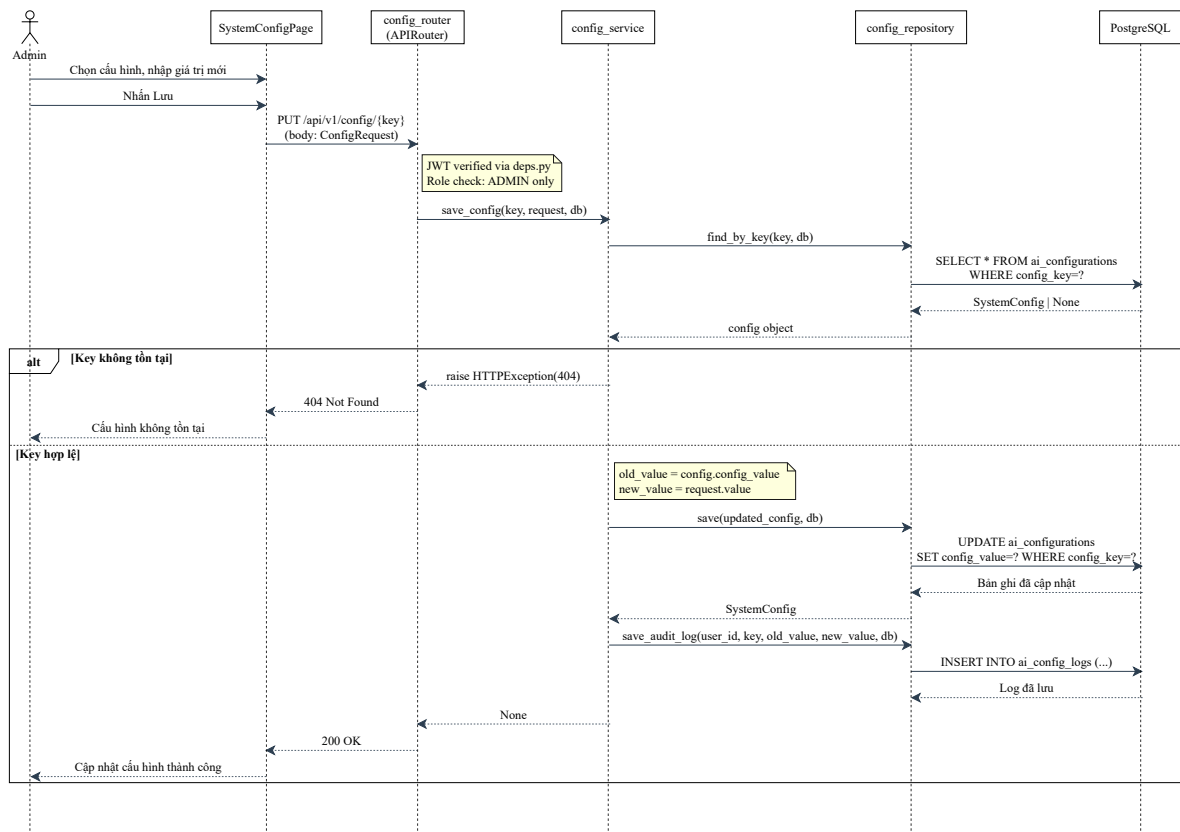
3.3 Update System Configuration (Cấu hình hệ thống)

1. Class Diagram



Hình 5: Class_Diagram - Cấu hình hệ thống

2. Sequence Diagram



Hình 6: Sequence_Diagram - Cấu hình hệ thống

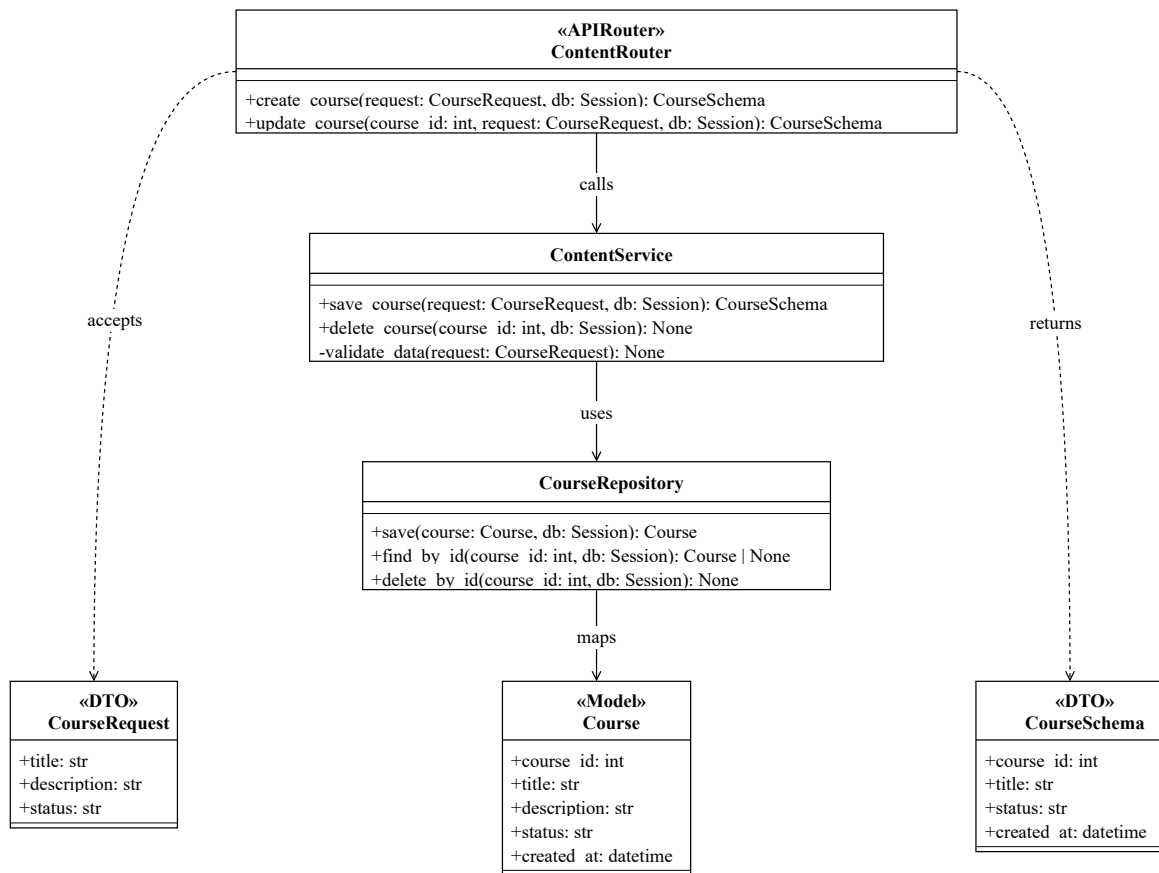
3. Class Specification

No	Class	Attributes / Methods	Description
1	SystemConfig	config_key: str, config_value: str, description: str, updated_at: datetime	SQLAlchemy Model lưu các cặp key-value cấu hình hệ thống/AI vào bảng ai_configurations. config_key là primary key. Mọi thay đổi phải ghi audit log theo BR-34.
2	ConfigRequest	value: str	Pydantic Schema đầu vào chứa giá trị mới cần cập nhật. Key được truyền qua path parameter /key, không nằm trong body.
3	ConfigSchema	key: str, value: str, description: str	Pydantic Schema đầu ra trả về thông tin cấu hình hiện tại cho admin xem xét.

No	Class	Attributes / Methods	Description
4	ConfigRouter	update_settings(key: str, request: ConfigRequest, db: Session): None, get_settings(db: Session): list[ConfigSchema]	FastAPI APIRouter xử lý PUT /config/{key} để cập nhật và GET /config để lấy toàn bộ cấu hình. key là path parameter được inject tự động bởi FastAPI.
5	ConfigService	save_config(key: str, request: ConfigRequest, db: Session): None, get_all_settings(db: Session): list[ConfigSchema]	Service kiểm tra key tồn tại, cập nhật giá trị và ghi audit log gồm: người thực hiện, hành động, giá trị cũ, giá trị mới và thời gian (BR-34, CR-05).
6	ConfigRepository	find_by_key(key: str, db: Session): SystemConfig None, save(config: SystemConfig, db: Session): SystemConfig, find_all(db: Session): list[SystemConfig]	Data access layer tìm cấu hình theo key, lưu thay đổi và lấy toàn bộ bản ghi cấu hình từ PostgreSQL.

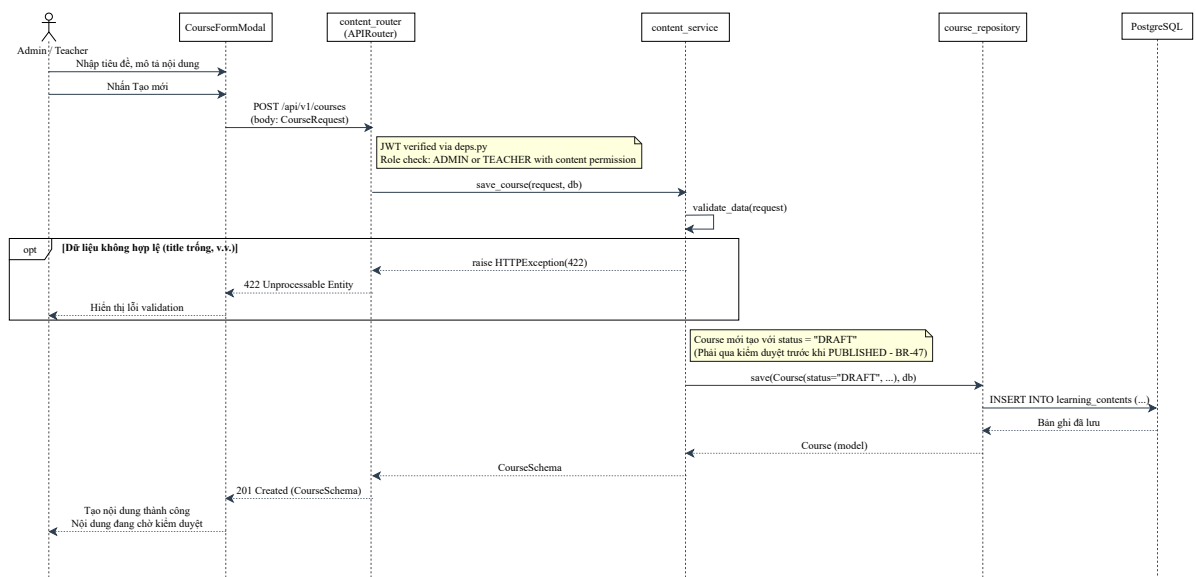
3.4 Manage Learning Content (Quản lý nội dung học)

1. Class Diagram



Hình 7: Class_Diagram - Quản lý nội dung học

2. Sequence Diagram



Hình 8: Sequence_Diagram - Quản lý nội dung học

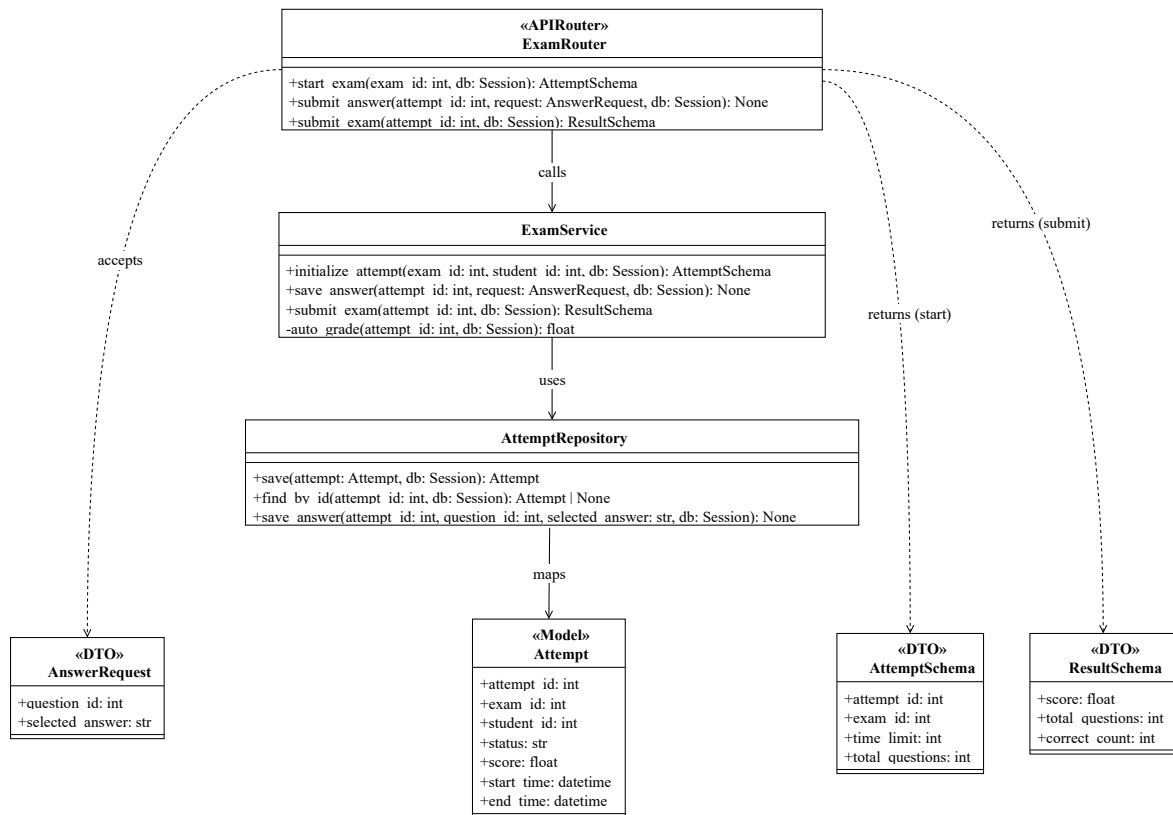
3. Class Specification

No	Class	Attributes / Methods	Description
1	Course	course_id: int, title: str, description: str, status: str {DRAFT PUBLISHED ARCHIVED}, created_at: datetime	SQLAlchemy Model đại diện cho khóa học/bài học trong bảng learning_contents. Nội dung chỉ được publish sau khi qua kiểm duyệt (BR-47, status = PUBLISHED).
2	CourseRequest	title: str, description: str, status: str	Pydantic Schema đầu vào dùng cho cả tạo mới và cập nhật nội dung học. Khi tạo mới, status mặc định là DRAFT.
3	CourseSchema	course_id: int, title: str, status: str, created_at: datetime	Pydantic Schema đầu ra trả về thông tin khóa học sau khi tạo hoặc cập nhật thành công.

No	Class	Attributes / Methods	Description
4	ContentRouter	create_course(request: CourseRequest, db: Session): CourseSchema, update_course(course_id: int, request: CourseRequest, db: Session): CourseSchema	FastAPI APIRouter xử lý POST /courses (tạo mới) và PUT /courses/{id} (cập nhật). Sequence Diagram minh họa luồng tạo mới; luồng cập nhật và xóa tương tự.
5	ContentService	save_course(request: CourseRequest, db: Session): CourseSchema, delete_course(course_id: int, db: Session): None, validate_data(request: CourseRequest): None	Service điều phối: kiểm tra dữ liệu hợp lệ, lưu và xóa nội dung học. Áp dụng BR-44 (chỉ vai trò có quyền mới quản lý được nội dung), BR-47 (status workflow).
6	CourseRepository	save(course: Course, db: Session): Course, find_by_id(course_id: int, db: Session): Course None, delete_by_id(course_id: int, db: Session): None	Data access layer thực hiện CRUD trên bảng learning_contents thông qua SQLAlchemy ORM.

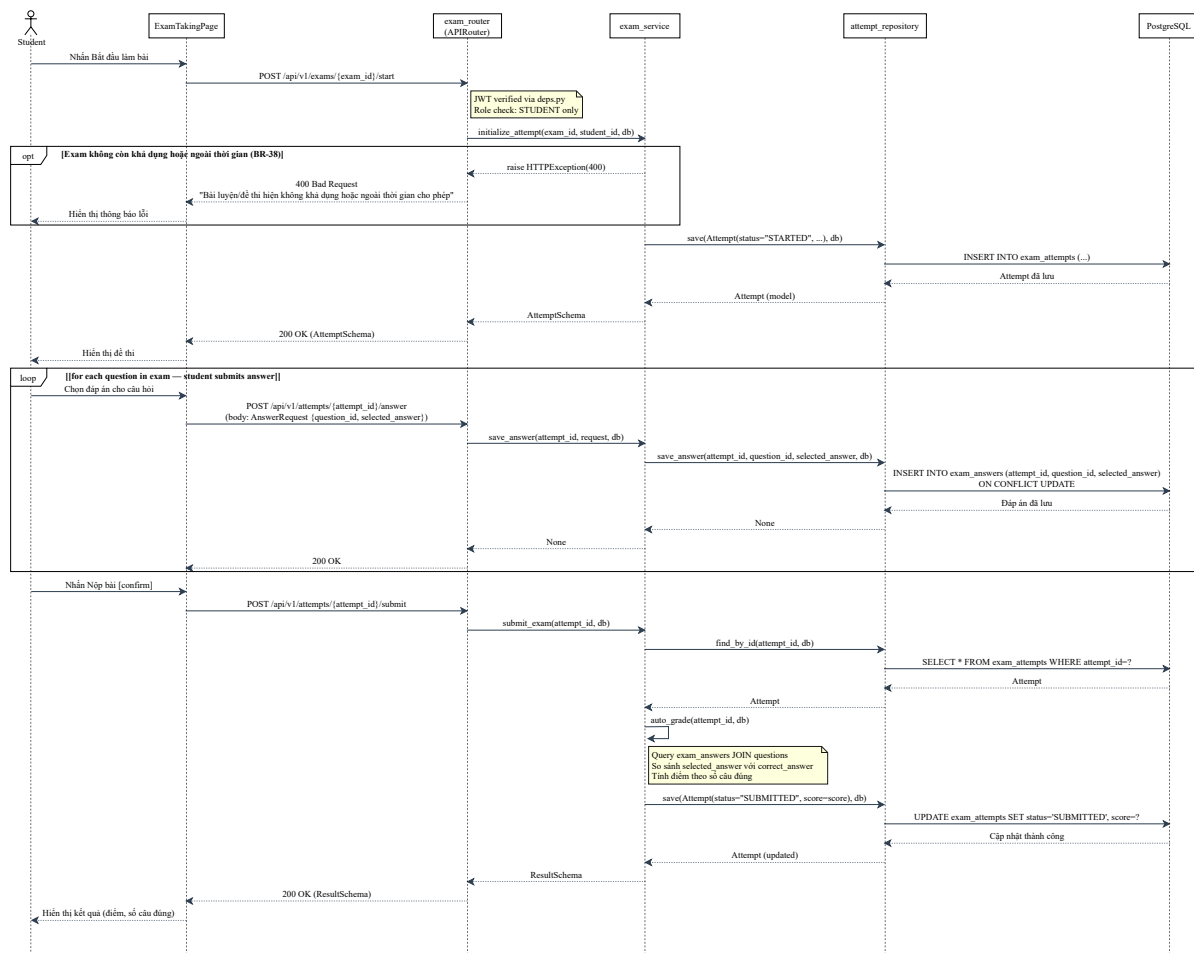
3.5 Take Competency Assessment (Làm bài thi ĐGNL)

1. Class Diagram



Hình 9: Class_Diagram - Làm bài thi ĐGNL

2. Sequence Diagram



Hình 10: Sequence_Diagram - Làm bài thi ĐGNL

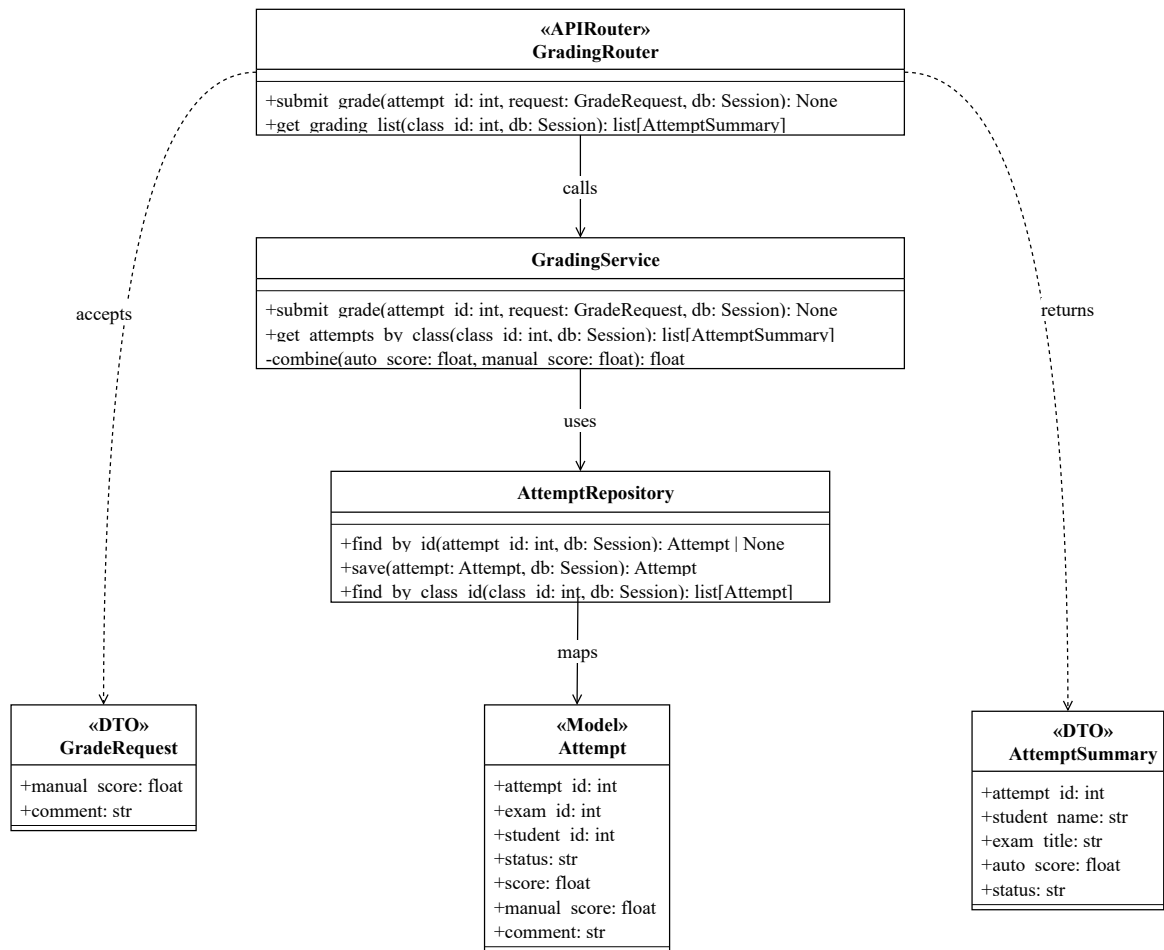
3. Class Specification

No	Class	Attributes / Methods	Description
1	Attempt	attempt_id: int, exam_id: int, student_id: int, status: str, score: float, start_time: datetime, end_time: datetime	SQLAlchemy Model lưu một lần làm bài của học sinh trong bảng exam_attempts. end_time dùng để kiểm tra hết hạn (BR-38). Status: STARTED / SUBMITTED / GRADED.

No	Class	Attributes / Methods	Description
2	AnswerRequest	question_id: int, selected_answer: str	Pydantic Schema đầu vào mang câu trả lời của học sinh cho một câu hỏi. question_id bắt buộc để định danh đúng bản ghi trong bảng exam_answers.
3	AttemptSchema	attempt_id: int, exam_id: int, time_limit: int, total_questions: int	Pydantic Schema đầu ra trả về khi bắt đầu làm bài, cung cấp thông tin cần thiết cho giao diện hiển thị đề thi.
4	ResultSchema	score: float, total_questions: int, correct_count: int	Pydantic Schema đầu ra trả về kết quả sau khi nộp bài và chấm điểm tự động hoàn tất.
5	ExamRouter	start_exam(exam_id: int, db: Session): AttemptSchema, submit_answer(attempt_id: int, request: AnswerRequest, db: Session): None, submit_exam(attempt_id: int, db: Session): ResultSchema	FastAPI APIRouter quản lý 3 giai đoạn của luồng làm bài: bắt đầu, lưu từng câu trả lời (loop) và nộp bài.
6	ExamService	initialize_attempt(exam_id: int, student_id: int, db: Session): AttemptSchema, save_answer(attempt_id: int, request: AnswerRequest, db: Session): None, submit_exam(attempt_id: int, db: Session): ResultSchema, auto_grade(attempt_id: int, db: Session): float	Service điều phối toàn bộ luồng thi: kiểm tra đề còn khả dụng (BR-38), tạo attempt mới, lưu đáp án, nộp bài và tính điểm tự động bằng cách so sánh với đáp án chuẩn.
7	AttemptRepository	save(attempt: Attempt, db: Session): Attempt, find_by_id(attempt_id: int, db: Session): Attempt None, save_answer(attempt_id: int, question_id: int, selected_answer: str, db: Session): None	Data access layer trên hai bảng: exam_attempts và exam_answers. Method save_answer nhận cả attempt_id và question_id để cập nhật đúng bản ghi (composite key).

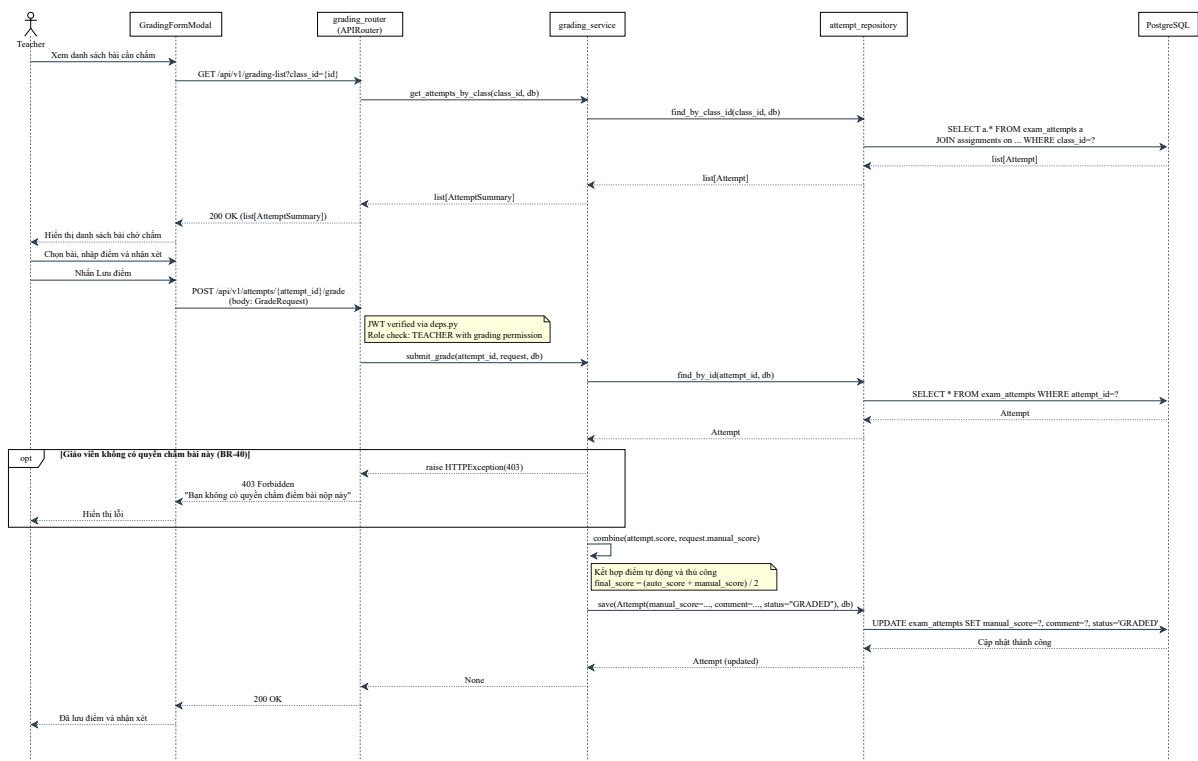
3.6 Grade Student Submission (Giáo viên chấm điểm)

1. Class Diagram



Hình 11: Class_Diagram - Giáo viên chấm điểm

2. Sequence Diagram



Hình 12: Sequence_Diagram - Giáo viên chấm điểm

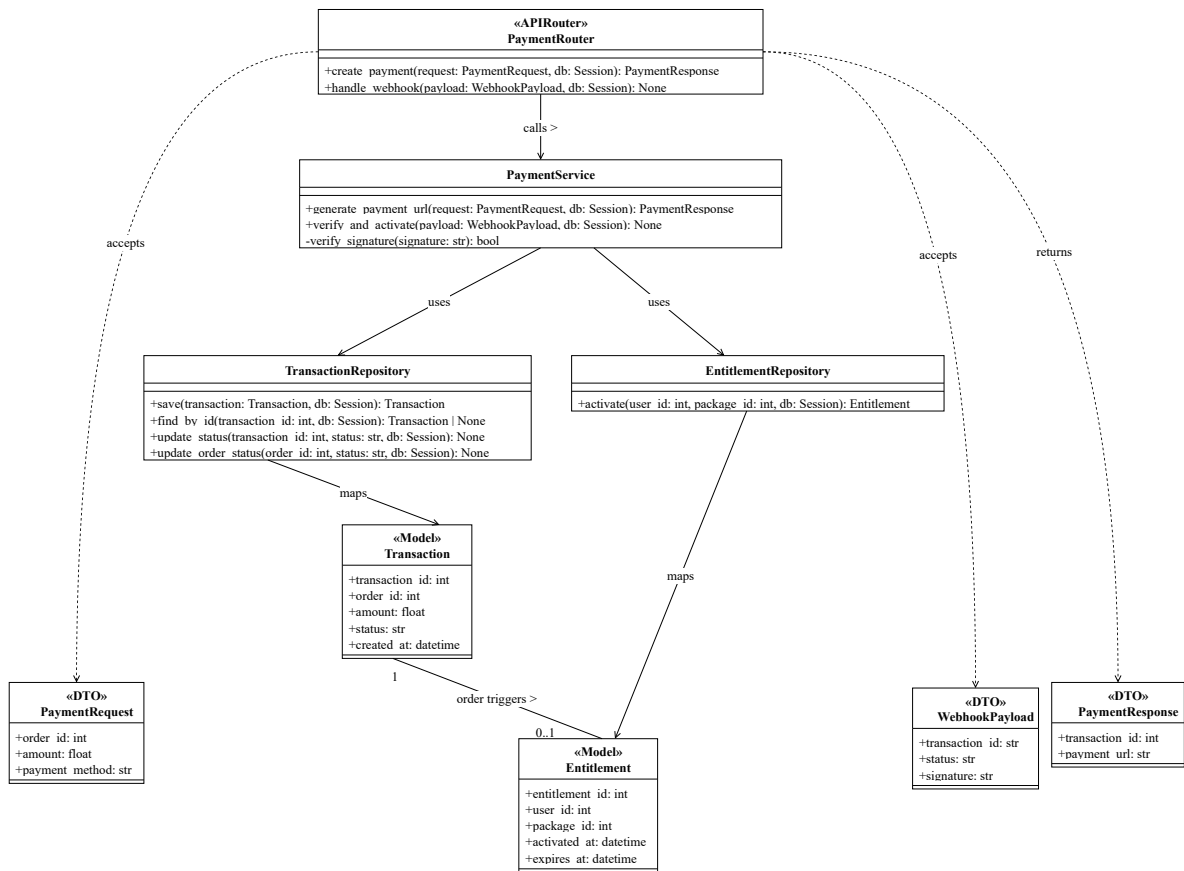
3. Class Specification

No	Class	Attributes / Methods	Description
1	Attempt	attempt_id: int, exam_id: int, student_id: int, status: str, score: float, manual_score: float, comment: str	SQLAlchemy Model lưu kết quả làm bài và thông tin chấm thủ công của giáo viên. manual_score và comment được cập nhật khi giáo viên chấm (BR-40).
2	GradeRequest	manual_score: float, comment: str	Pydantic Schema đầu vào chứa điểm chấm thủ công và nhận xét của giáo viên cho một bài làm.
3	AttemptSummary	attempt_id: int, student_name: str, exam_title: str, auto_score: float, status: str	Pydantic Schema đầu ra dùng trong danh sách chờ chấm, giúp giáo viên biết tên học sinh, tên bài thi và điểm tự động.

No	Class	Attributes / Methods	Description
4	GradingRouter	submit_grade(attempt_id: int, request: GradeRequest, db: Session): None, get_grading_list(class_id: int, db: Session): list[AttemptSummary]	FastAPI APIRouter xử lý POST /attempts/{id}/grade để lưu điểm và GET /grading-list để lấy danh sách cần chấm.
5	GradingService	submit_grade(attempt_id: int, request: GradeRequest, db: Session): None, get_attempts_by_class(class_id: int, db: Session): list[AttemptSummary], combine(auto_score: float, manual_score: float): float	Service kiểm tra giáo viên có quyền chấm bài đó không (BR-40), kết hợp điểm tự động và điểm thủ công qua combine(), rồi lưu kết quả cuối.
6	AttemptRepository	find_by_id(attempt_id: int, db: Session): Attempt None, save(attempt: Attempt, db: Session): Attempt, find_by_class_id(class_id: int, db: Session): list[Attempt]	Data access layer truy vấn bài làm theo ID và theo lớp từ bảng exam_attempts trong PostgreSQL.

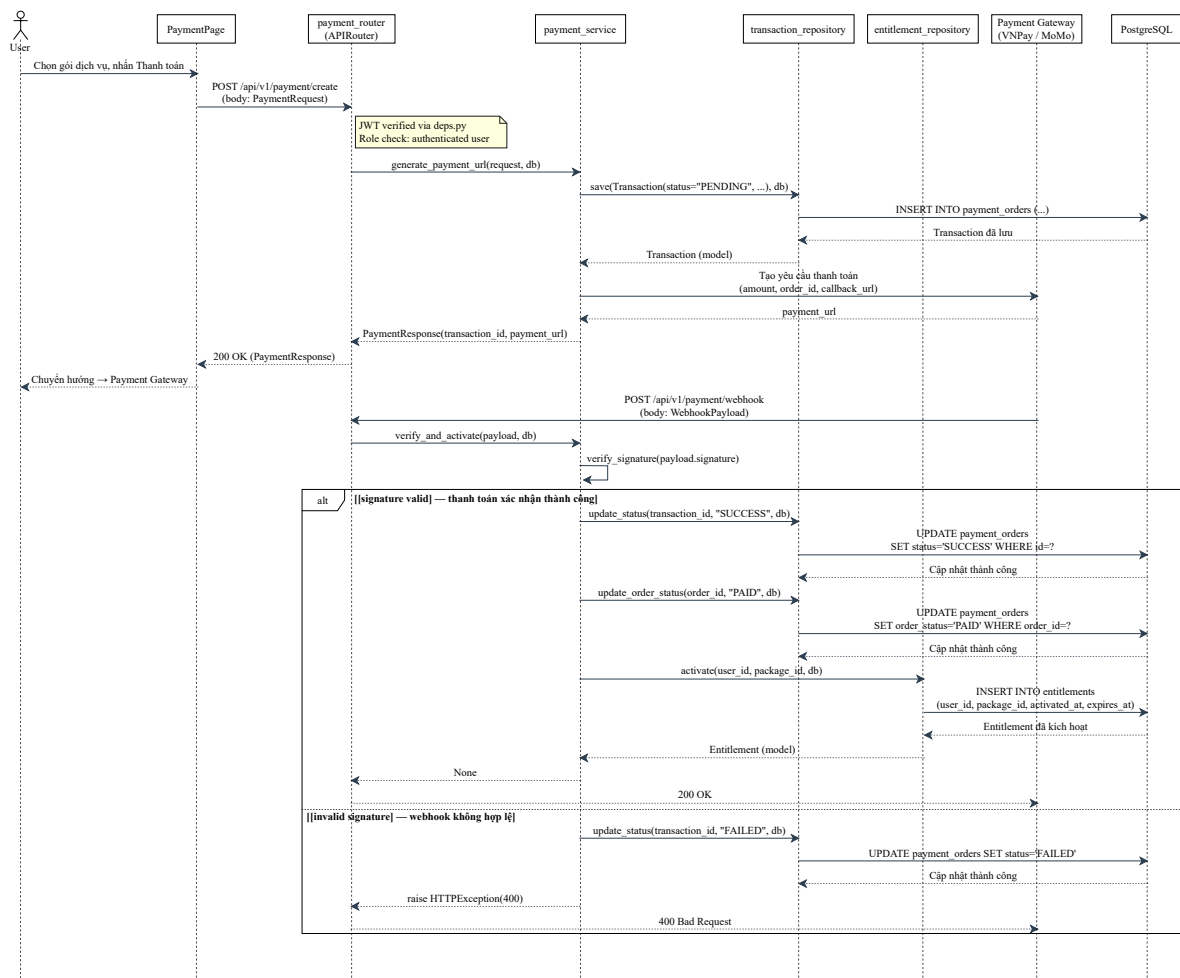
3.7 Process Online Payment (Thanh toán trực tuyến)

1. Class Diagram



Hình 13: Class_Diagram - Thanh toán trực tuyến

2. Sequence Diagram



Hình 14: Sequence_Diagram - Thanh toán trực tuyến

3. Class Specification

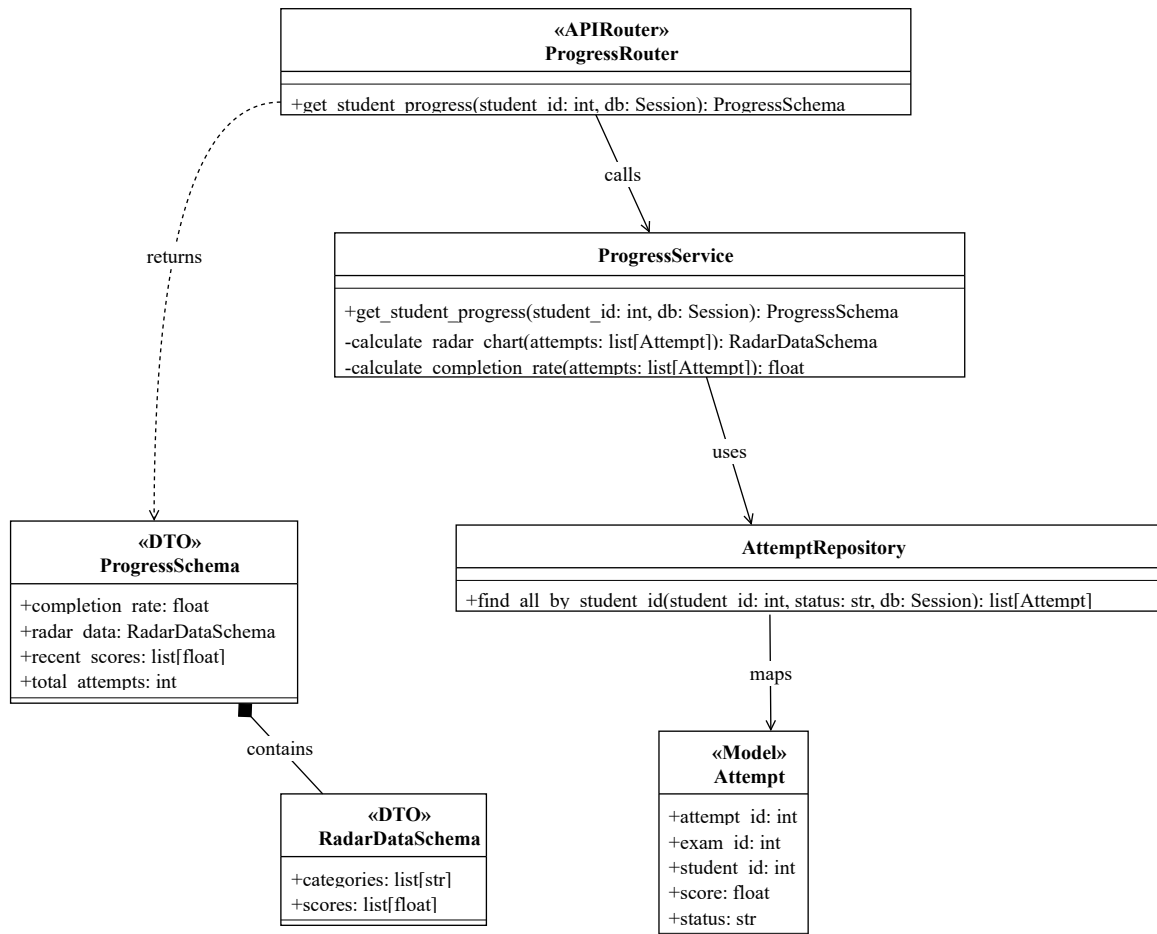
No	Class	Attributes / Methods	Description
1	Transaction	transaction_id: int, order_id: int, amount: float, status: str, created_at: datetime	SQLAlchemy Model lưu giao dịch thanh toán trong bảng payment_orders. order_id là foreign key liên kết với đơn hàng. Status: PENDING / SUCCESS / FAILED.
2	Entitlement	entitlement_id: int, user_id: int, package_id: int, activated_at: datetime, expires_at: datetime	SQLAlchemy Model lưu quyền lợi được kích hoạt sau thanh toán thành công (bảng entitlements). Chỉ được tạo khi giao dịch được xác nhận (BR-53).

No	Class	Attributes / Methods	Description
3	PaymentRequest	order_id: int, amount: float, payment_method: str	Pydantic Schema đầu vào chứa thông tin đơn hàng khi người dùng tiến hành thanh toán.
4	PaymentResponse	transaction_id: int, payment_url: str	Pydantic Schema đầu ra trả về URL thanh toán để frontend chuyển hướng người dùng đến cổng thanh toán.
5	WebhookPayload	transaction_id: str, status: str, signature: str	Pydantic Schema đầu vào từ callback webhook của cổng thanh toán (VNPay/MoMo). signature dùng để xác thực tính hợp lệ của webhook.
6	PaymentRouter	create_payment(request: PaymentRequest, db: Session): PaymentResponse, handle_webhook(payload: WebhookPayload, db: Session): None	FastAPI APIRouter xử lý POST /payment/create để tạo URL thanh toán và POST /payment/webhook để nhận callback từ cổng thanh toán.
7	PaymentService	generate_payment_url(request: PaymentRequest, db: Session): PaymentResponse, verify_and_activate(payload: WebhookPayload, db: Session): None, verify_signature(signature: str): bool	Service điều phối: tạo giao dịch, gọi Payment Adapter để lấy URL, xác thực chữ ký webhook, cập nhật trạng thái đơn hàng và kích hoạt quyền lợi (BR-53, BR-54).
8	TransactionRepository	save(transaction: Transaction, db: Session): Transaction, find_by_id(transaction_id: int, db: Session): Transaction None, update_status(transaction_id: int, status: str, db: Session): None, update_order_status(order_id: int, status: str, db: Session): None	Data access layer quản lý bảng payment_orders. update_order_status() cập nhật trạng thái đơn hàng khi webhook xác nhận thành công.

No	Class	Attributes / Methods	Description
9	EntitlementRepository	<pre> activate(user_id: int, package_id: int, db: Session): Entitlement </pre>	Data access layer kích hoạt quyền lợi người dùng trong bảng entitlements sau khi thanh toán được xác nhận.

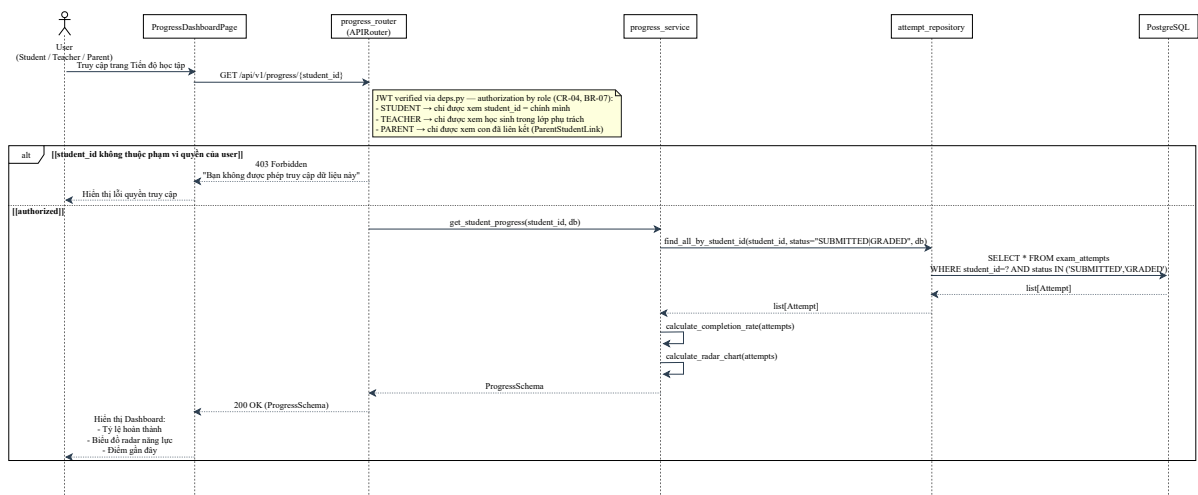
3.8 Track Learning Progress (Theo dõi tiến độ học tập)

1. Class Diagram



Hình 15: Class_Diagram - Theo dõi tiến độ học tập

2. Sequence Diagram



Hình 16: Sequence_Diagram - Theo dõi tiến độ học tập

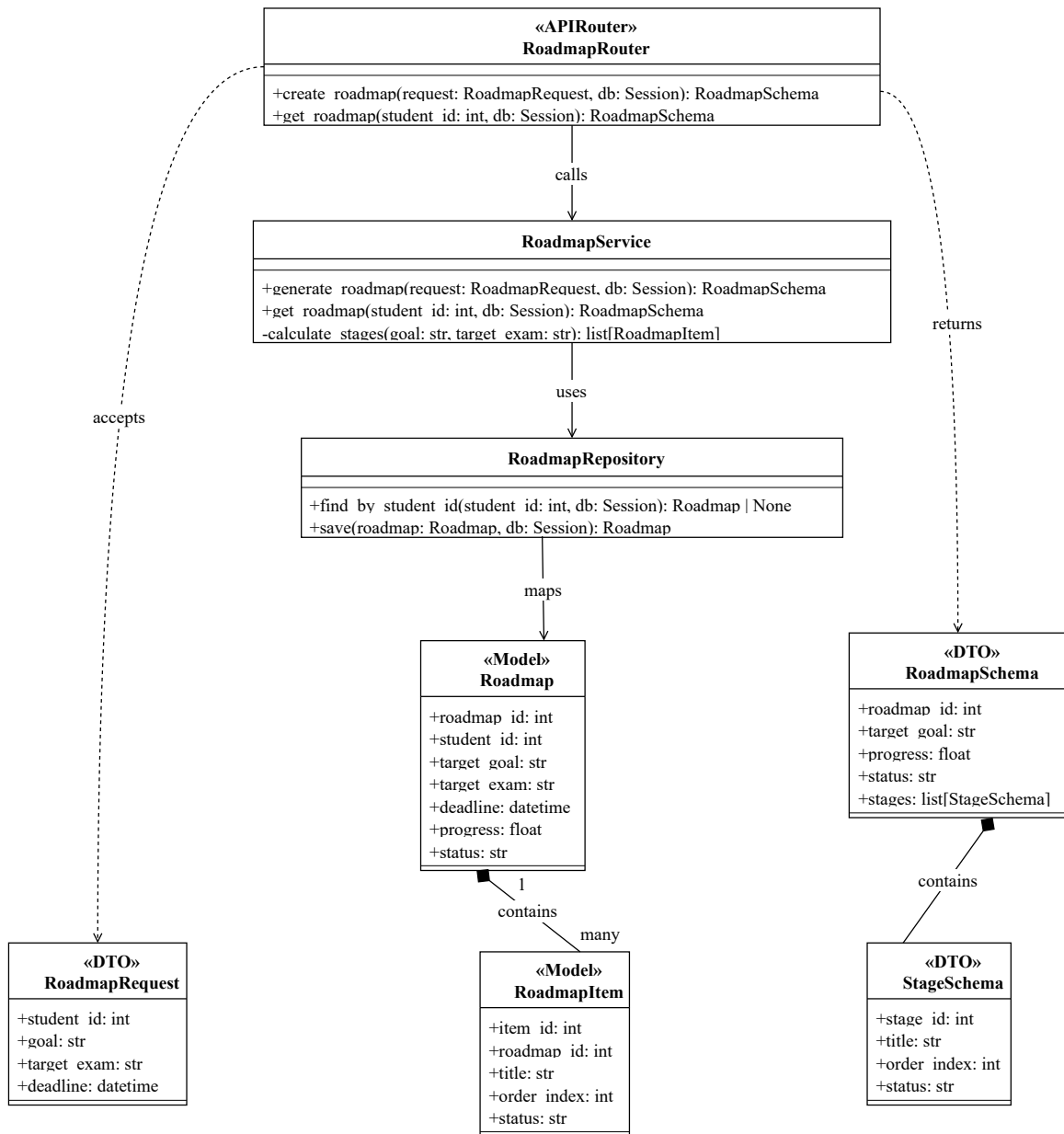
3. Class Specification

No	Class	Attributes / Methods	Description
1	Attempt	attempt_id: int, exam_id: int, student_id: int, score: float, status: str	SQLAlchemy Model lưu kết quả làm bài, là nguồn dữ liệu chính để tính toán tiến độ và biểu đồ radar năng lực.
2	RadarDataSchema	categories: list[str], scores: list[float]	Pydantic Schema đầu ra chứa dữ liệu radar chart phân tích năng lực theo từng chủ đề/môn.
3	ProgressSchema	completion_rate: float, radar_data: RadarDataSchema, recent_scores: list[float], total_attempts: int	Pydantic Schema đầu ra tổng hợp tỷ lệ hoàn thành, biểu đồ radar và lịch sử điểm gần nhất cho Dashboard.
4	ProgressRouter	get_student_progress(student_id: int, db: Session): ProgressSchema	FastAPI APIRouter xử lý GET /progress/{student_id}. Authorization được kiểm tra qua deps.py: Student chỉ xem của chính mình, Teacher xem lớp phụ trách, Parent xem con đã liên kết (CR-04, BR-07).

No	Class	Attributes / Methods	Description
5	ProgressService	<code>get_student_progress(student_id: int, db: Session): ProgressSchema,</code> <code>calculate_radar_chart(attempts: list[Attempt]): RadarDataSchema,</code> <code>calculate_completion_rate(attempts: list[Attempt]): float</code>	Service nhận danh sách attempts từ Repository rồi tính toán tỷ lệ hoàn thành và biểu đồ radar. Không truy vấn database trực tiếp (phải qua Repository).
6	AttemptRepository	<code>find_all_by_student_id(student_id: int, status: str, db: Session): list[Attempt]</code>	Data access layer lấy tất cả bài làm của học sinh từ bảng exam_attempts. Hỗ trợ filter theo status để chỉ tính các bài đã SUBMITTED/GRADED.

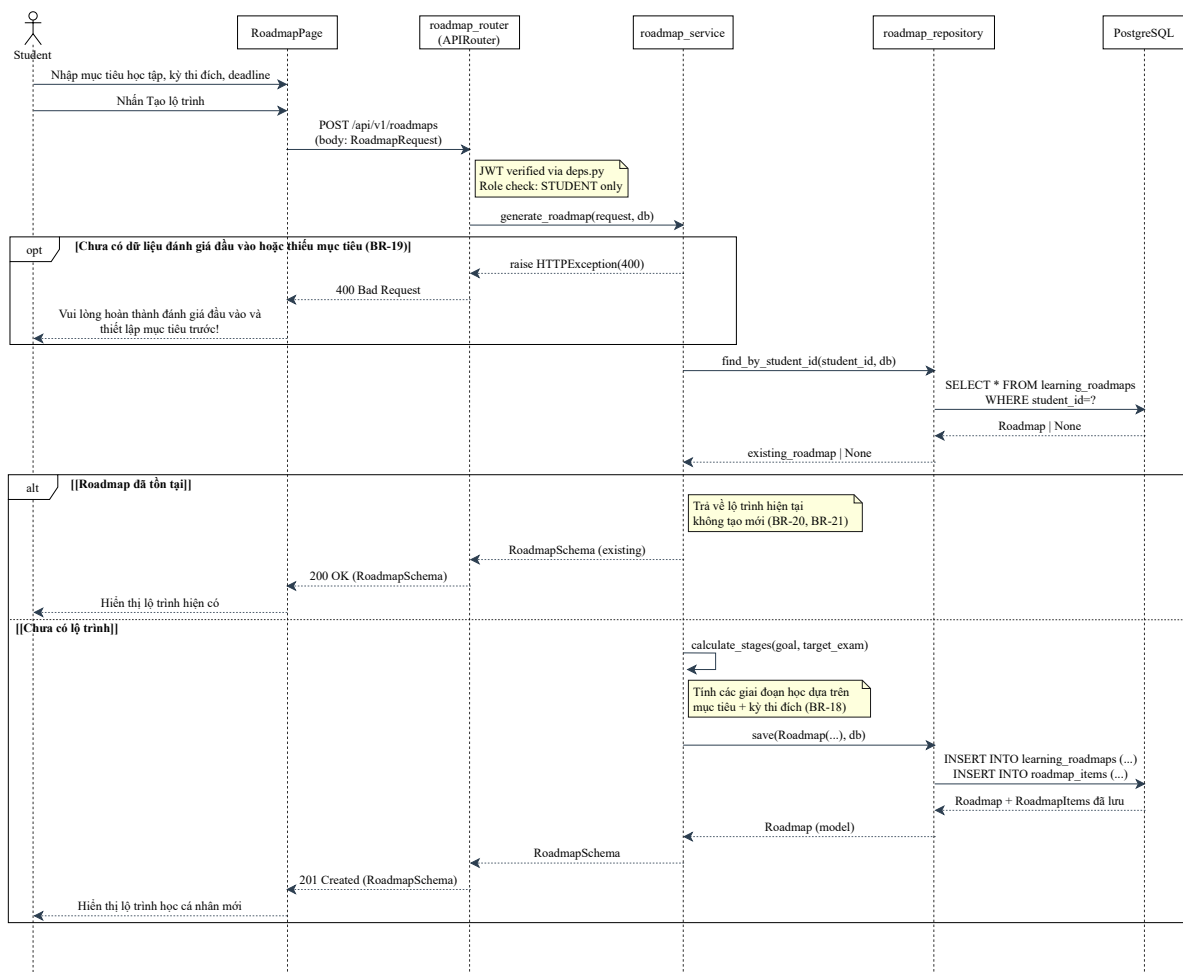
3.9 Create Personalized Roadmap (Lộ trình học cá nhân)

1. Class Diagram



Hình 17: Class_Diagram - Lộ trình học cá nhân

2. Sequence Diagram



Hình 18: Sequence_Diagram - Lộ trình học cá nhân

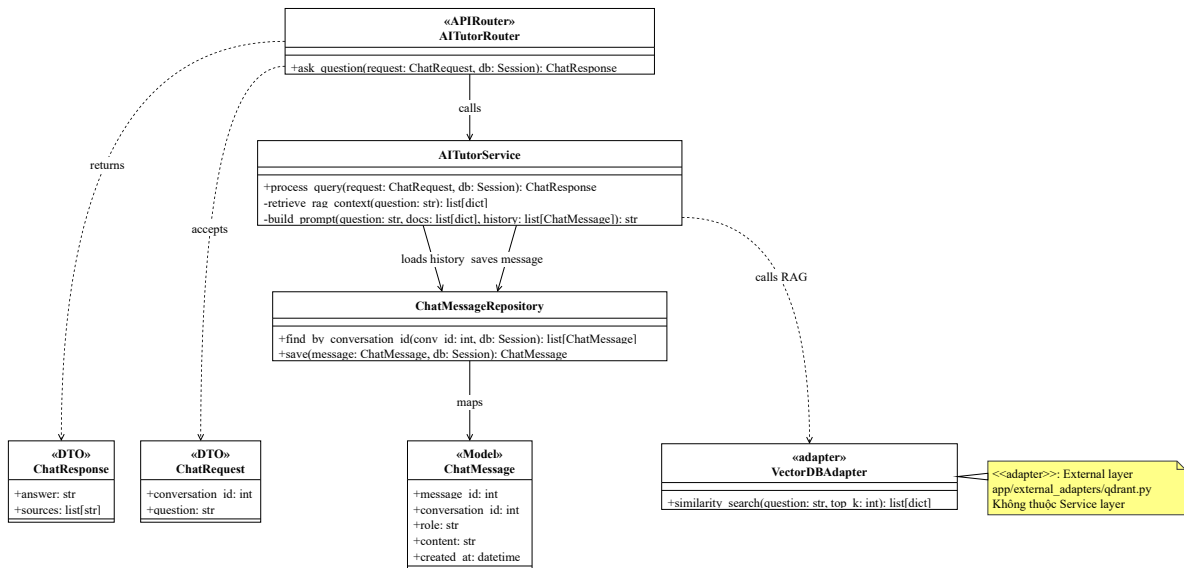
3. Class Specification

No	Class	Attributes / Methods	Description
1	Roadmap	roadmap_id: int, student_id: int, target_goal: str, target_exam: str, deadline: datetime, progress: float, status: str	SQLAlchemy Model lưu lộ trình học cá nhân hóa trong bảng learning_roadmaps. Gắn với đúng người học (BR-20); không được thay đổi lịch sử đã hoàn thành (BR-21).
2	RoadmapItem	item_id: int, roadmap_id: int, title: str, order_index: int, status: str	SQLAlchemy Model đại diện cho từng bước/bài học trong lộ trình, lưu trong bảng roadmap_items.

No	Class	Attributes / Methods	Description
3	RoadmapRequest	student_id: int, goal: str, target_exam: str, deadline: datetime	Pydantic Schema đầu vào từ học sinh khi yêu cầu tạo lộ trình. Cần có đủ dữ liệu đánh giá đầu vào trước khi tạo (BR-19).
4	StageSchema	stage_id: int, title: str, order_index: int, status: str	Pydantic Schema đầu ra biểu diễn từng giai đoạn trong lộ trình để frontend hiển thị.
5	RoadmapSchema	roadmap_id: int, target_goal: str, progress: float, status: str, stages: list[StageSchema]	Pydantic Schema đầu ra tổng hợp toàn bộ lộ trình bao gồm danh sách các giai đoạn học.
6	RoadmapRouter	create_roadmap(request: RoadmapRequest, db: Session): RoadmapSchema, get_roadmap(student_id: int, db: Session): RoadmapSchema	FastAPI APIRouter xử lý POST /roadmaps (tạo mới) và GET /roadmaps/{student_id} (xem lộ trình hiện tại).
7	RoadmapService	generate_roadmap(request: RoadmapRequest, db: Session): RoadmapSchema, get_roadmap(student_id: int, db: Session): RoadmapSchema, calculate_stages(goal: str, target_exam: str): list[RoadmapItem]	Service kiểm tra điều kiện đủ dữ liệu (BR-19), tạo hoặc lấy lộ trình hiện có, và tính toán các giai đoạn dựa trên mục tiêu và kỳ thi đích (BR-18).
8	RoadmapRepository	find_by_student_id(student_id: int, db: Session): Roadmap None, save(roadmap: Roadmap, db: Session): Roadmap	Data access layer truy vấn và lưu lộ trình học trong bảng learning_roadmaps và roadmap_items.

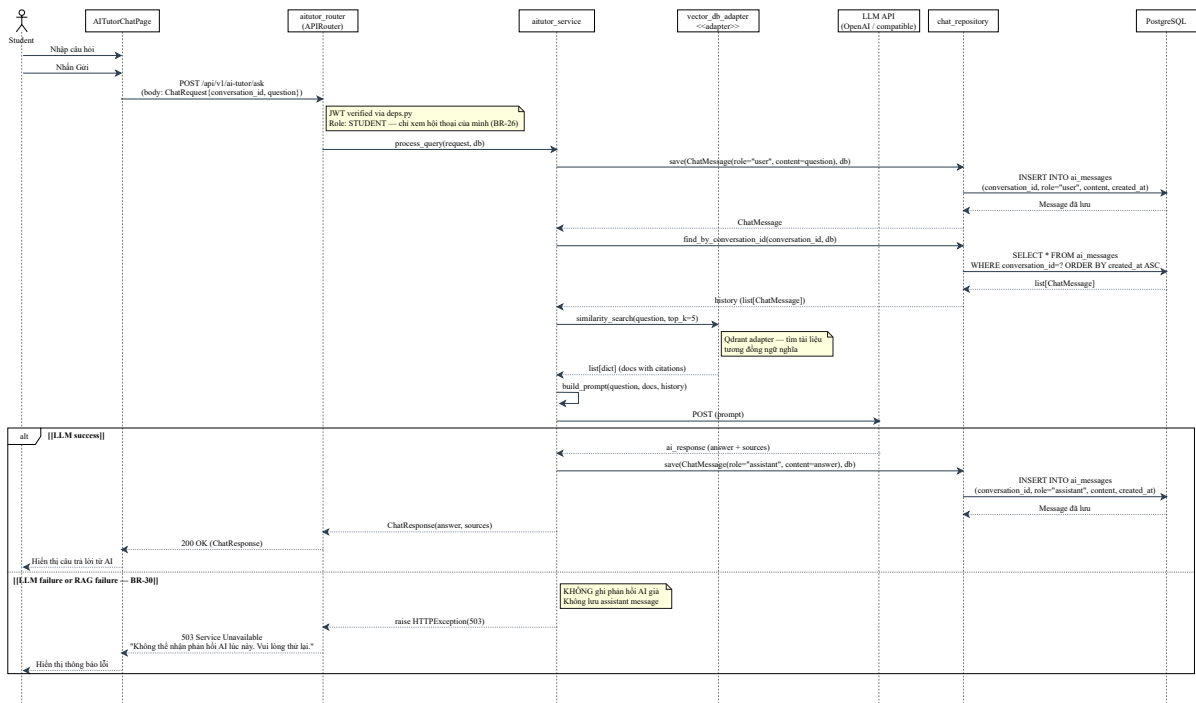
3.10 Ask AI Tutor (Trợ lý AI)

1. Class Diagram



Hình 19: Class_Diagram - Trợ lý AI

2. Sequence Diagram



Hình 20: Sequence_Diagram - Trợ lý AI

3. Class Specification

No	Class	Attributes / Methods	Description
1	ChatMessage	message_id: int, conversation_id: int, role: str {user assistant}, content: str, created_at: datetime	SQLAlchemy Model lưu từng tin nhắn trong bảng ai_messages. role phân biệt người gửi (BR-32). created_at đảm bảo thứ tự lịch sử (BR-32).
2	ChatRequest	conversation_id: int, question: str	Pydantic Schema đầu vào chứa câu hỏi của học sinh và ID hội thoại để tải lịch sử ngữ cảnh.
3	ChatResponse	answer: str, sources: list[str]	Pydantic Schema đầu ra chứa câu trả lời từ AI và danh sách tài liệu tham khảo (citations) từ RAG pipeline.
4	AITutorRouter	ask_question(request: ChatRequest, db: Session): ChatResponse	FastAPI APIRouter xử lý POST /ai-tutor/ask. Chỉ học sinh sở hữu hội thoại mới được gửi câu hỏi (BR-26).
5	AITutorService	process_query(request: ChatRequest, db: Session): ChatResponse, retrieve_rag_context(question: str): list[dict], build_prompt(question: str, docs: list[dict], history: list[ChatMessage]): str	Service điều phối toàn bộ RAG pipeline: tải lịch sử hội thoại, truy xuất ngữ cảnh từ Vector DB, xây dựng prompt, gọi LLM và lưu kết quả. Xử lý lỗi RAG/LLM theo BR-30.
6	VectorDBAdapter	similarity_search(question: str, top_k: int): list[dict]	External adapter tích hợp Qdrant Vector DB để tìm kiếm tài liệu liên quan theo độ tương đồng ngữ nghĩa. Thuộc tầng app/external_adapters/, không phải Service layer.
7	ChatMessageRepository	find_by_conversation_id(conv_id: int, db: Session): list[ChatMessage], save(message: ChatMessage, db: Session): ChatMessage	Data access layer truy vấn lịch sử hội thoại (phải được gọi TRƯỚC build_prompt) và lưu tin nhắn mới vào bảng ai_messages.